

Exercise sheet 1

Foundations – computers, algorithms, and your first program

Not graded, not handed in. Do them anyway. This is where the learning happens! Solutions are published after we discuss them in class.

You do not need a computer for this sheet. Every exercise is written to be done on paper. If you have a laptop, use it *after* you have written your answer down. The prediction is the exercise, the machine is only the marker.

Part A — Algorithms ---

Pen and paper. No laptop. Work in pairs where you can.

A1 *Write an algorithm*

Write an algorithm that a stranger could follow without asking you a single question. Pick one:

- making an espresso with a moka pot;
- getting from this room to the nearest exit;
- deciding whether a year is a leap year.

State the **input** and the **output** explicitly. Number every step.

Then swap with your neighbour and try to break their algorithm. Find one step that two people could carry out differently.

A2 *Find the ambiguity*

Five ambiguities are hiding in this algorithm. Find them.

INPUT: a list of exam marks

1. Look at the marks
2. Throw away the bad ones
3. Add up what is left
4. Divide by the number of marks
5. Round the result

OUTPUT: the average mark

A3 *Trace it*

```
READ n

SET total TO 0
SET i TO 1

WHILE i <= n:
    SET total TO total + i
    SET i TO i + 1

PRINT total
```

- a) Trace it for $n = 4$. Draw a table with one column per variable and one row per step.
- b) What does this algorithm compute, in one sentence?
- c) What does it print for $n = 0$? Is that the answer you would want?

A4 *Find the broken case*

```
READ a, b

IF a > b THEN
    PRINT a
ELSE
    PRINT b
```

- a) It is supposed to print the larger of the two numbers. For which inputs does it not do what a careful person would want?
- b) Fix it.

A5 *Where it never stops*

```
SET x TO 10

WHILE x > 0:
    PRINT x
    SET x TO x + 1
```

- a) What was the author probably trying to do?
- b) Why does this never stop?
- c) Fix it with a one-character change.

A6 *Flowchart back to pseudocode*

```

( START )
|
READ temp
|
< temp > 30 ? >  --yes--> PRINT "hot"
|
no
|
< temp < 10 ? >  --yes--> PRINT "cold"
|
no
|
PRINT "fine"
|
( END )

```

- Write this as pseudocode.
- What does it print for $\text{temp} = 30$? For $\text{temp} = 10$?
- Is that sensible? If not, say what you would change.

A7 *Design one*

A shop gives a discount:

- 10% if you spend more than €50;
- 20% if you spend more than €100;
- nothing otherwise.

- Write the algorithm. Input: the amount spent. Output: the amount to pay.
- Trace it for €40, €50, €75, €100 and €150.
- One of those five values is a trap. Which one, and why?

Part B — Your first Python program

Everyone works on paper. If you have a laptop, check your answers afterwards.

Reference — everything you need for Part B

Printing

```
print("Hello, world")      -> Hello, world
print("Hello", "world")   -> Hello world
print("2 + 2 =", 2 + 2)   -> 2 + 2 = 4
print()                   -> an empty line
# a line starting with # is ignored by Python
```

Arithmetic

operator	meaning	example	result
+ - *	add, subtract, multiply	7 * 3	21
/	divide — always gives a decimal	6 / 3	3.0
//	floor division — the whole part, rounded down	7 // 3	2
%	modulo — the remainder	7 % 3	1
**	power	7 ** 3	343
()	evaluated first	(7 + 3) * 2	20

Multiplication and division happen before addition and subtraction.

The four errors

error	what it means
SyntaxError	Python could not read your program. Nothing ran
IndentationError	leading spaces that nothing above justifies
NameError	a name Python has never been given. Names are case-sensitive
TypeError	an operation that makes no sense for those kinds of value

B1 Hello, world, by hand

Write out, by hand, the one line that makes Python display Hello, world. Then answer in one sentence each:

- What is print — and what do the parentheses do?
- What would happen if you left out the quotation marks?
- Where does the output actually appear?

B2 *Predict the output*

Write your prediction next to each line before reading any further.

	expression		expression
1	2 ** 10	5	(2 + 3) * 4
2	17 % 5	6	10 / 2
3	17 // 5	7	7 // 2.0
4	2 + 3 * 4	8	-7 // 2

Numbers 6, 7 and 8 are the interesting ones. When we run them at the front, write down in one sentence what surprised you.

B3 *Six broken lines*

For each line: **name the error**, say **why** in a few words, and **write the fix**.

```
# 1
print("The answer is" 42)

# 2
Print("hello")

# 3
print("2 + 2 = " + 2 + 2)

# 4
x = 10
print(X)

# 5
print("first")
    print("second")

# 6
print("she said "hello" and left")
```

Then pick the one you are least sure about. We will run that one on the projector.

B4 *Write a small receipt*

Write out the Python — by hand — that produces exactly this output. The numbers must be **computed by Python**, not typed in as text:

```
Coffee    x 3 = 4.5
Cornetto  x 2 = 3.0
Total           7.5
```

A coffee costs 1.50 and a cornetto 1.50.

B5 *A calculator problem*

You bought 3 items at €12.40 each and paid with a €50 note.

- Write the Python that prints the change, computed rather than typed in.
- Add two more lines: how many €5 notes fit into that change, and what is left over.
- One of the three numbers this prints will look wrong. Predict which one, and say why you think so. (You will not be able to explain it fully yet — guessing is the point.)

Hint for (b): two of the operators in the reference table above do exactly this.

B6 *Notebook or script?*

Nothing to run. Answer in one or two sentences:

- In a notebook, a cell containing just `7 + 3` displays `10`. The same line in a `.py` script displays nothing at all. Why?
- Name one practical reason to prefer a script over a notebook for a real program.

If you finish early

Do not start the next sheet. Do this instead:

- Help someone near you. Explaining is how you find out what you only half-know.
- Take your algorithm from **A1** and try to write it in Python. You will not have the tools yet — that is the point. Write down precisely which step you could not express. Keep the note. We will come back to it.

Before lecture 2 — at home

Install Python on your own machine. Pick one:

how	where
A Python + VS Code, with the Python and Jupyter extensions	python.org · code.visualstudio.com
B Google Colab — nothing to install, runs in the browser	colab.research.google.com
C Miniforge — conda, preconfigured to conda-forge	github.com/conda-forge/miniforge
D Anaconda	anaconda.com/download

Check it worked, in a terminal:

```
python --version          # or: python3 --version (macOS) . py --version (Windows)
python -c "print('ready')"
```

You want 3.10 or higher.

- On macOS the command is `python3`, not `python` — macOS has shipped no bare python since 2022. Inside an activated conda or virtual environment both names work.

- On Windows, tick **Add python.exe to PATH** during installation. Forgetting this is the most common single failure — typing python will open the Microsoft Store instead.
- If it fights you, come and find me – ten minutes after a lecture solves almost all of it.

Then run `01-foundations-demo.ipynb` from the `lectures/` folder of the course repository and work through it. It contains everything we did at the front today, so you can repeat it at your own pace.